

Instantly Shareable Playtest

Implementation Specification: Playtest × xCloud streaming

Comprehensive Review — Annotated Copy with Gap Highlights & Cross-Repo Verification

Melanie Chen · Microsoft Xbox Intern

Faithful copy of *Draft v2 · 2026-06-03* (27 pp.), with highlighted additions for missing / incomplete areas, and findings **verified against the four implementation repos** (xbet, SAGE, Partner Registry, Content Ingestion).

Generated review · 2026-06-10

How to read this comprehensive review

This document is an **exact copy** of the existing *Instantly Shareable Playtest — Implementation Specification (Draft v2, 2026-06-03)*. The original prose is reproduced section-by-section and unchanged. Two kinds of annotation are inserted in place:

⚠ REVIEW · LEGEND

Yellow = gap callouts

- **MISSING** — a TOC (or cross-referenced) section with no body in the source.
- **INCOMPLETE** — a section explicitly unfinished (e.g. `FINALIZE THIS SECTION`).
- **DRIFT** — spec content that no longer matches the shipped code.
- **RECOVERED** — suggested fill-in reconstructed from the earlier reviewed draft / project brief.

✓ VERIFIED IN CODE

Blue = verified against the four repos

Blue boxes carry facts confirmed by reading the actual branches, with `file:line` citations. Repos reviewed:

- **xbet / xPlaytest** — `Xbox.Xbet.Service` @ t-melanichen/playtest-ingestion-payload-builder
- **SAGE** — `services.serviceapigateway` @ t-melanichen/sage-playtest-ingestion-routes
- **Partner Registry** — `services.partnerregistry` @ t-melanichen/remove-getpackagesourceid
- **Content Ingestion** — `services.contentingestion` @ t-melanichen/playtest-title-ingestion-workflow

0. Cross-Repo Verification Summary *(new — not in the source spec)*

The highest-signal findings from reading the four branches. Details are inlined at the relevant sections.

⚠ REVIEW · DRIFT

BUG RISK — Platform value mismatch between xbet and the xCloud receiver

xbet builds the payload with `Platform = PlaytestConstants.DefaultPlatform = "WINDOWS.DESKTOP"` (`Xbox.Xbet.Service: PlaytestConstants.cs:12; PlaytestIngestionPayloadBuilder.cs:94`). The receiver's `StoreAsset.Validate()` throws `Unsupported platform unless Platform == ServerPlatform.PC (the Id "PC") or ServerPlatform.Xbox` (`services.contentingestion: ContentIngestion.Contracts.External/Workflow/StoreAsset.cs:35-46`). `"WINDOWS.DESKTOP"` does not appear anywhere in `services.contentingestion`. As written, a real payload would fail receiver validation.

Action: map xbet's platform constant to the receiver's `ServerPlatform.PC` id (`"PC"`) before sending, or have the receiver accept `WINDOWS.DESKTOP`. Confirm the exact `ServerPlatform.PC` string in `Services.Common`.

⚠ REVIEW · DRIFT

DELETE / by-playtest teardown route is documented but not implemented

The spec (§6.2.1) and SAGE's OpenAPI advertise `DELETE .../playtestingestion/by-playtest/{playtestId}`, but it exists in **no** running surface: **not in SAGE's route config** (`services.serviceapigateway: appsettings.xcloud.json:456-487` lists only `POST + GET`) and **not in the receiver controller** (`services.contentingestion: WorkflowsControllerV3.cs:48-59` has `POST + GET` only). PT6 teardown is therefore unimplemented.

⚠ REVIEW · DRIFT

Route name drift: 'playtestingestion' vs 'playtesttitleingestion'

The shipped route is `playtesttitleingestion` (`SAGE appsettings.xcloud.json:467-480; content-ingestion WorkflowsControllerV3.cs:48-59`), but the spec text and SAGE's own OpenAPI use `playtestingestion` (no 'title'). Pick one spelling and align the spec, OpenAPI, SAGE config, and the receiver controller.

✓ VERIFIED IN CODE

Confirmed: the §5.1 contract really does carry PartnerId + PlaytestProductId

Both the sender and receiver contracts include them as required: `Xbox.Xbet.Service PlaytestIngestionJobParameters.cs:29-38` and `services.contentingestion ContentCatalog.Common.Contracts/Workflows/PlaytestTitleIngestion.cs (JobParameters fields + Validate)`. xbet sets `PartnerId = SellerId` and `PlaytestProductId = PlaytestGeneratedXProductBigId` (`PlaytestIngestionPayloadBuilder.cs:110-111; PlaytestIngestionPayloadBuilderTests.cs:72-73`).

✓ VERIFIED IN CODE

Confirmed: single-PR offering + title; real polling policies; PC readiness shortcut

- Receiver builds one `PlaytestRequest` (offering + bound title) and calls `partnerRegistryClient.ConfigurePlaytestAsync(request)` (`PlaytestTitleIngestionWorkflow.cs:220, 307-330`); Partner Registry then submits offering and title in one ADO PR via `BulkEditAsync([offering, title])` (`PartnerRegistryService/Processors/PlaytestProcessor.cs:101-107; Registry/RegistryProcessor{T}.cs:133-185`).
- Polling uses configured policies `AssetIngestionJobPollingPolicy` / `FirstInstallPollingPolicy` (each `Interval + Timeout; MaxAttemptCount = Timeout/Interval + 1`) (`IngestionWorkflowSettings.cs:8-25; IngestionWorkflowBase.cs:24-31`).
- v1 PC titles **skip the first-install poll** — readiness is gated on offering configuration, with a `TODO` for a real PC readiness check (`PlaytestTitleIngestionWorkflow.cs:239-246`).

Table of Contents

- 1. What We Are Building
- 2. End-to-End Flow
- 3. Design Decisions & Tradeoffs
- 4. Team Responsibilities
- 5. Data Contract
- 6. REST API Contract
- 7. Known Blockers
- 8. Resolved Decisions
- 9. Open Questions & Recommendations
- 10. Success Criteria (P0)
- 11. Milestones & Foundational Demo
- 12. Glossary — italicized terms are defined here

⚠ REVIEW · DRIFT

Table of Contents vs. body numbering mismatch

The TOC lists **12 numbered sections ending at ‘12. Glossary’**, but the body contains an unnumbered **‘12. Next Steps’** and presents the glossary as **‘Appendix A: Glossary’**. **‘Next Steps’** never appears in the TOC, and the glossary is numbered two ways.

Fix: renumber Next Steps as §12 and Glossary as Appendix A (drop from numbered TOC), or keep Glossary as §12 and add Next Steps as §13. Make TOC + body agree.

1. What We Are Building

Today, xPlaytest lets game creators distribute private pre-release builds to testers as downloadable installs through *Garrison*. Modern games are hundreds of gigabytes — downloads are slow, hardware provisioning does not scale, and shipping physical devices to testers risks unreleased code leaking.

The Instantly Shareable Playtest project wires xPlaytest into Xbox Cloud Gaming’s streaming pipeline. When a creator publishes a playtest with the **Enable Cloud Streaming** option checked, their RETAIL-signed PC build is ingested into xCloud and becomes streamable to authorized testers via a one-click shareable link — in addition to still being available for download.

This is purely additive. Creators who do not opt in get the existing download-only flow. Creators who do opt in get both download and streaming for the same playtest.

2. End-to-End Flow

2.1 Diagram Flow

⚠ REVIEW · MISSING

Sequence diagram not embedded

The §2.1 heading has **no diagram** on the page (reviewer: “Please add the diagrams directly”). The four-phase flow is described in §2.2, but there is no visual sequence (xPlaytest → SAGE → services.contentingestion → Partner Registry → install pipeline) nor swim-lane ownership view. **Fix:** embed the publish→handoff→ingest→streamable diagram inline.

2.2 Table Flow

A streaming-enabled playtest goes through four phases. The first phase runs entirely on xPlaytest; the middle two run on xCloud; the fourth is a status reconciliation across both.

#	Phase	What Happens	Owner
1	Publish Trigger	Creator clicks Publish in Partner Center. <code>PublishPlaytest</code> validates the playtest and hands off to the existing <code>XPackagePlaytestPublishWorkflow</code> . The workflow runs the existing state machine (<code>FetchingContentIds</code> → <code>StartingContentSubmissionJob</code> → <code>PollingContentSubmissionJob</code> → <code>PlaytestProductCreation</code>). We insert a new state, <code>XCloudIngestionTrigger</code> , immediately after <code>PlaytestProductCreation</code> and before <code>SuccessCompletion</code> .	xPlaytest
2	xCloud Handoff	The new state builds a single <code>PlaytestIngestionJobParameters</code> payload (see §5) from data the workflow already resolved for XProduct, and POSTs it to SAGE, which proxies cross-tenant to services.contentingestion. The response is 202 Accepted with a jobId. Ingestion failure does not block the download playtest from publishing.	xPlaytest → xCloud
3	xCloud Ingestion	A new <code>PlaytestTitleIngestionWorkflow</code> runs inside services.contentingestion. It (1) validates inbound parameters; (2) chains the existing <code>AssetIngestionWorkflow</code> using the	xCloud

	Workflow	supplied StoreAsset; (3) writes the playtest offering and attaches its title in a single Partner Registry PR; (4) polls the install pipeline for capacity.	
4	Streamable	When the xCloud workflow reaches a terminal state, xPlaytest learns about it by polling the SAGE status endpoint with the jobId. On success, xPlaytest sets <code>PlaytestStatus = StreamingReady</code> and surfaces the launch URL <code>https://play.xbox.com/play/launch/{productId}?offeringId=xpt-{id}</code> .	xCloud → xPlaytest

✓ VERIFIED IN CODE

Verified workflow stages on the receiver

The shipped `PlaytestTitleIngestionWorkflow` stages are: `ValidateParameters` → `TriggerAssetIngestionAsync` (Suspend/Checkpoint) → `HandleAssetIngestionResult / ...Timeout` → `CreatePackageAsync` → `ConfigureOfferingAsync` → `PollFirstInstallAsync` → `NotifyReadyAsync / NotifyInstallNotFoundAsync` → `ConcludeWorkflow` (`PlaytestTitleIngestionWorkflow.cs:64-339`). Matches phase 3 above.

3. Design Decisions & Tradeoffs

3.0 Streaming requires a resolvable Xbox Title ID

Decision: The “Enable Cloud Streaming” option is offered only when the selected package is Xbox Live-configured — i.e., it has a resolvable numeric Xbox Title ID, enforced at publish.

Rationale: XboxTitleId is required by the xCloud contract for games (`StoreAsset.Validate()` rejects null/zero) and Playtest’s `PlaytestProductDocumentBuilder` currently hardcodes it empty. The ID is not minted by Partner Center on publish; for a normal title it comes from Big Cat. Since xPlaytest populates the playtest’s XProduct document and XProduct only supports lookup by product Big ID, the title ID has to be read from the parent/source product. Because our team owns XOrc we can source it there given the parent product ID; XOrc returns the package family name, application ID, and title ID.

Trade-off Accepted: Developers whose title has never been published cannot stream their playtest in v1.

⚠ REVIEW · MISSING

XboxTitleId resolver is still an open blocker, not a solved design

The shipped `PlaytestIngestionPayloadBuilder.Build(...)` takes `uint xboxTitleId` as a caller-supplied parameter and states resolving it is “intentionally out of scope here” (`PlaytestIngestionPayloadBuilder.cs:50-58, 96`); `Build_ZeroXboxTitleId_Throws` confirms a zero id is rejected (`PlaytestIngestionPayloadBuilderTests.cs:101-108`). The repo’s `Blockers/XboxLiveIDs/` file is empty. **Add** a concrete owner + task for the XOrc lookup and the publish-time failure behavior when XOrc returns nothing.

3.1 Manual PR Approval

Constraint: Every Partner Registry write creates an ADO PR. SFI policy forbids self-approval in production.

v1 mitigation:

1. Create a Partner Registry endpoint that creates an offering and attaches a title in a single PR instead of two sequential writes.
2. Accept that someone on the xCloud side approves those PRs out-of-band.

v1.1: If the payload carried the parent product ID, automated pre-approval could match and gate on it. Requires an SFI exception and is GSS’s call.

✓ VERIFIED IN CODE

Verified: single-PR create-offering + attach-title exists

Partner Registry’s `PlaytestProcessor` writes offering and title together in one PR — “Offering is listed first so it exists in the same PR before the title that references it” — via `registryProcessor.BulkEditAsync(new IRegistryObject[] { offering, title }, prTitle, user)` (`PlaytestProcessor.cs:101-107`; `RegistryProcessor{T}.cs:133-185`). Registry objects are git/ADO-backed JSON, not SQL (`RegistryProcessor{T}.cs:45-71`).

3.2 Authorization Gate at the Offering vs Title Level

Decision: Playtest offerings are private. Access is gated by a new `AllowedDnaGroups` field on the offering's `PlayerAuthorizationOptions`. The title inside the offering bypasses the per-title entitlement check.

Rationale: Centralizing access control at the offering layer gives a single firm `GsToken.DnaGroups` check at offering entry, after which any title in that offering is reachable. This is the pattern *TestX* already uses. One-offering-per-playtest means no risk of exposing an unrelated title.

Tradeoff: Skipping the per-title entitlement check slightly relaxes defense-in-depth; mitigated because the offering is private and DNA-group-locked.

External dependency: Partner Registry only stores the allow-list; the actual `GsToken.DnaGroups` ↔ `AllowedDnaGroups` check happens in the streaming-token / Bayside path.

✓ VERIFIED IN CODE

Verified: `AllowedDnaGroups` + `AllowedSandboxId` on `PlayerAuthorizationOptions`

`PlayerAuthorizationOptions` has `ICollection<string> AllowedDnaGroups` and `Id? AllowedSandboxId` (`PartnerRegistryClient/Contracts/PlayerAuthorizationOptions.cs:69-80`), surfaced on the offering via `OfferingV2.AuthorizationOptions` (`OfferingV2.cs:48-52`) and populated by `PlaytestProcessor` (`PlaytestProcessor.cs:78-83`). Write routes are S2S app-id allow-listed (`Startup.cs:384-398`; `RegistryManagementConfig.cs:8-13`).

3.3 Pick One DNA group as the Effective “Flight” for Installation

Decision: When the xCloud install pipeline needs a single flight identifier, the workflow passes the **lexicographically smallest DNA group GUID** from `AllowedDnaGroups`.

Rationale: A playtest can have multiple DNA groups, each a SUCU “flight,” all pointing to the same build version. SUCU expects exactly one flight ID; a deterministic lex-smallest choice gives idempotent re-runs.

Caveat — not “sticky”: lex-smallest is recomputed each resolve, so adding/removing GUIDs can change the resolved flight. Acceptable in v1.

This is an install-time lookup rule only; audience auth still checks the full `AllowedDnaGroups` set.

3.4 Streaming is Optional and in Addition to the Standard Playtest

Decision: And, not or. A streaming-enabled publish also produces a downloadable playtest. Streaming is gated behind an “Enable Cloud Streaming” checkbox.

Rationale: Treating them as mutually exclusive would force a hard cut-over; layering streaming on top keeps existing behavior unchanged for non-opt-in creators.

3.5 One playtest → One title → One offering

Decision: Each published playtest gets exactly one xCloud offering with exactly one title. The offering id is `xpt-{shortId(PlaytestId)}`.

Rationale: Offering ↔ audience must be 1:1 to keep authorization crisp. Clustering a developer's playtests into one offering is future scope.

Tradeoff: Many active playtests create many offerings; the id scheme is stable enough to evolve later.

3.6 PC-first, Xbox Console Deferred

Decision: v1 scope is PC builds only.

Rationale: The data path is set up for PC (`WINDOWS.DESKTOP` platform constant, PC-only `ContainsMSIXVCByDeviceFamily`). Console adds Xbox-specific Title ID and package handling.

AumID resolution path: AumID is only consumed by the server's upload-login telemetry, so a constant default unblocks v1; v1.1 plumbs the real ApplicationId from the AppX manifest.

Risk — PC support in the install poll path: the existing `TitleIngestionWorker.PollFirstInstallAsync` hardcodes Xbox-only allocator parameters (`ServerType.XboxV3SeriesS`, `XBOX_*` pool ids). XC6 must audit + fork the install poll path so PC playtests pick a PC pool. Tracked under XC6 and Known Blockers §7.

✓ VERIFIED IN CODE

Verified: receiver short-circuits the PC install poll in v1

The receiver does not run the Xbox install poll for PC playtests: "v1 Treat a PC title as ready once the offering is configured" with a TODO: add a proper PC readiness check ([PlaytestTitleIngestionWorkflow.cs:239-246](#)). So the §3.6 risk is sidestepped (not solved) for v1 PC.

3.7 Naming and Identifiers

Decision: Offering id is `xpt-{shortId}` (opaque). Title id is `<alphanumOnly(PlaytestName)>{PlaytestGeneratedXProductBigId}`.

Rationale: Offering id must be ≤ 32 chars; a full GUID is 36, so we hash/truncate. Appending the product id gives uniqueness when a developer reuses a game name.

Tradeoff documented: opaque ids aren't human-identifiable; mitigated by surfacing the friendly name in UX we control.

⚠ REVIEW - DRIFT

Offering-id key: PlaytestId vs the stable product id

§3.5/§3.7 derive the offering id from `PlaytestId`, but the receiver builds it from the **product id** (`xpt{PlaytestProductId}`) which must be stable across republishes ([Xbox.Xbet.Service.PlaytestIngestionJobParameters.cs:32-38](#)), and Partner Registry centralizes the offering id / package source id on the request (`services.partnerregistry branch commit d17cf3ed 'Centralize playtest offering id and package source id on PlaytestRequest'`). **Align the spec on the stable product id** (see §8 Resolved Decisions).

3.8 Bespoke endpoint vs. generic "CreateOfferingFromStoreData"

Decision: Bespoke endpoint. **Rationale:** A generic surface is the future direction but out of scope; the bespoke shape encodes playtest-specific defaults (sandbox = RETAIL, isPrivate = true) server-side and keeps the contract small. **Tradeoff:** some boilerplate duplication later; migrating is a controlled refactor.

3.9 Cross-tenant S2S strategy

Short term: xPlaytest (MSFTGreen tenant) → xCloud SAGE (Corp tenant) using the Green→Corp cross-tenant auth scheme (PR #15715445). **Long term:** xCloud migrates to PME; PME does not accept Green

callers out of the box. The GSS platform team owns the long-term solution.

✓ VERIFIED IN CODE

Verified: SAGE allow-lists the xPlaytest identities on the playtest routes

AuthorizedClientAppIds on the playtest route includes the xPlaytest SPI (d60e3360-830b-4a09-b4bb-0f759ca83e06) and MSI (97a28a30-87b4-45e9-a6c6-c23a183a72ba) (appsettings.xcloud.json:471-484). Note: a UseCrossTenantAuth flag on this route was **not** found in config — confirm cross-tenant enforcement is actually wired (XC2).

3.10 Polling vs Callback for Ingestion Completion

Decision: xPlaytest learns about completion by polling GET /.../playtestingestion/{jobId} on SAGE.
Rationale: SAGE is a request/response proxy with no callback/event-hub layer; adding one is out of scope. xPlaytest already polls inside its workflow, and services.contentingestion exposes job status through WorkflowsControllerV3 ; SAGE just forwards.

Polling strategy: FINALIZE THIS SECTION

⚠ REVIEW · INCOMPLETE

§3.10 Polling strategy left unfinished (“FINALIZE THIS SECTION”)

This placeholder is load-bearing: it’s cross-referenced by PT5 (“exponential backoff (§3.1)” — wrong ref, should be §3.10), §6.2, and the “60-min foreground deadline.”
Recommended xPlaytest-side fill-in: poll with exponential backoff (e.g. 5s start, ×1.5, cap 60s) until a terminal status; 60-min foreground deadline, then hand to a background reconciliation job (PT5); persist jobId on PublishedPlaytestEntity ; 5xx retryable, 404 terminal-fail. Fix the PT5 cross-ref (§3.1 → §3.10).

✓ VERIFIED IN CODE

Verified receiver-side polling policies (anchor the numbers to these)

The receiver workflow polls using configured policies, not magic numbers:
AssetVersionIngestionJobPollingPolicy , AssetIngestionJobPollingPolicy , FirstInstallPollingPolicy — each with Interval + Timeout (IngestionWorkflowSettings.cs:8-25), with MaxAttemptCount = Timeout/Interval + 1 (IngestionWorkflowBase.cs:24-31). Asset ingestion uses Suspend(... Timeout = AssetIngestionJobPollingPolicy.Timeout).Checkpoint() (PlaytestTitleIngestionWorkflow.cs:100-111). The finalized §3.10 numbers should be consistent with these.

3.11 API versioning strategy

Decision: Two layers, two conventions.

Layer	Routes	Versioning
External (xPlaytest → SAGE)	POST /v3/playtest/playtesttitleingestion , GET /.../{jobId}	OpenAPI info.version (1.0.0); reserve /v2 for breaking changes.

Internal (SAGE → services.contentingestion)	POST /v3/workflows/playtesttitleingestion, GET /.../{jobId}	URL-versioned (/v3), matching WorkflowsControllerV3 .
---	---	---

Evolution rules: prefer additive optional fields; document required vs optional in `openapi/playtest-ingestion.yaml`; a breaking external change bumps to `/v2`. Expected future addition (non-breaking): `CallbackUrl` once cross-tenant messaging exists.

✓ VERIFIED IN CODE

Verified SAGE route config (and a doc drift)

SAGE forwards `v3/playtest/playtesttitleingestion` (POST) and `.../{jobId}` (GET) to backend `v3/workflows/playtesttitleingestion` on `contentingestion-svc` ([appsettings.xcloud.json:456-487](#)). The branch also touches `openapi/playtest-ingestion.yaml`, `ARCHITECTURE.md`, `SPEC.md` — the OpenAPI still says `playtestingestion` and includes a DELETE route (see \$0 callouts). Reconcile.

3.12 Streamable Playtest End Time

Constraint: Streaming-enabled playtests must carry an explicit `PlaytestEndDate` rather than the default `DateTime.MaxValue`. Download-only playtests keep the no-expiry default.

Rationale: a streaming offering is live infrastructure tying up xCloud capacity; a bounded end date lets it auto-expire and be reclaimed.

Tradeoff: diverges from the download flow and adds one required field for streaming creators.

✓ VERIFIED IN CODE

Verified: future end date is enforced offline

The builder rejects a null/default/past expiration before any send ([PlaytestIngestionPayloadBuilder.cs:149-152](#); [PlaytestIngestionPayloadBuilderTests.cs:119-126](#) 'Build_PastExpiration_Throws').

4. Team Responsibilities

4.1 xPlaytest deliverables

Note: presented in call sequence order.

ID	Task	Notes	SWAG
PT1	Integrate xCloud trigger into XPackagePlaytestPublishWorkflow	New <code>XCloudIngestionTriggerState</code> . Failure does NOT block publish. Streaming-publish validators require <code>PlaytestEndDate</code> + seller/title allow-list gate.	6
PT2	Construct StoreAsset from resolved product/package data	Mostly a projection of in-memory state. Two gaps: XboxTitleId resolver (P0 blocker) and AumId default-or-resolver.	4
PT3	Build offering config from audience data	DNA group IDs already resolved via GMS (<code>FlightIds</code>). Package into payload.	1
PT4	Send ingestion request via SAGE	New <code>XCloudIngestionClient</code> . Cross-tenant S2S token; 202 + jobld handling; persist jobld. Blocked on SAGE allow-list (XC2).	3
PT5	Handle completion signal	Polling loop with backoff (\$3.10) + background reconciliation past the 60-min deadline. Transition to <code>StreamingReady</code> . Launch-URL builder. New <code>StreamingReady</code> enum value.	6
PT6	Handle lifecycle updates	Re-trigger on audience/expiry/republish; teardown on delete via the new SAGE DELETE route.	5
xPlaytest total			25

⚠ REVIEW · STATUS

PT4–PT6 are not yet implemented on the xbet branch

The reviewed branch contains only the shared contracts + the offline `PlaytestIngestionPayloadBuilder` + tests (`git diff: 5 files under PlayTest.Shared/Contracts and XPackageWorkflow`). There is **no** `XCloudIngestionClient`, polling loop, launch-URL builder, or `StreamingReady` enum value yet — PT4/PT5/PT6 remain open. PT2/PT3 (the builder) are done. A status column would make this explicit.

4.2 xCloud deliverables

ID	Service	Task	Notes	SWAG
XC1	SAGE	Add routes POST/GET (+DELETE)	Config-driven in <code>appsettings.xcloud.json</code> ; returns whatever <code>services.contentingestion</code> returns.	3
XC2	SAGE	Allow-list the Playtest S2S identity	<code>AuthorizedClientAppIds</code> + cross-tenant on both routes.	1
XC3	<code>services.contentingestion</code>	<code>PlaytestTitleIngestionWorkflow</code>	validate → asset ingest → create package → configure offering → poll	9

			install → terminal state; new contract + V3 routes.	
XC4	services.partnerregistry	Extend PlayerAuthorizationOptions (AllowedDnaGroups + AllowedSandboxId)	New fields on OfferingV2; single-PR bulk edit; content-ingestion SPN allow-listing.	5
XC5	Login / Offering auth	DNA-group check at offering access	GsToken.DnaGroups $\supseteq$ one of AllowedDnaGroups → grant. Bypass per-title check.	5
XC6	Install pipeline	Single-flight-id + PC fork	Lex-smallest GUID; PC pool / ServerType branch.	4
xCloud total				27

✓ VERIFIED IN CODE

Verified status of XC3/XC4 (and the no-DELETE gap)

XC3 workflow + V3 POST/GET routes exist ([WorkflowsControllerV3.cs:48-59](#); [PlaytestTitleIngestionWorkflow.cs](#)); XC4 fields + single-PR bulk edit + S2S allow-list exist ([PlayerAuthorizationOptions.cs:69-80](#); [PlaytestProcessor.cs:101-107](#); [Startup.cs:384-398](#)). The DELETE/by-playtest tombstone route (needed by PT6) is absent on both SAGE and the receiver.

5. Data Contract

5.1 PlaytestIngestionJobParameters (xPlaytest builds, xCloud consumes)

Field	Type	Source on xPlaytest	Required	Notes
PlaytestId	Guid	parameters.PlaytestId	yes	Stable id used to derive offering identity.
AllowedDnaGroups	string[]	PlaytestPublishJobParameters.FlightIds (resolved via GMS)	yes	xCloud wraps each in <code>XusAudience(g, AllowedSandboxId)</code> .
ExpirationTime	DateTime	PlaytestPublishJobParameters.PlaytestEndDate	yes (streaming)	Must be a future UTC timestamp.
AllowedSandboxId	string	parameters.Sandbox ?? "RETAIL"	no	Defaults to RETAIL.
StoreAsset	object	See §5.2	yes	Existing xCloud contract.
IsGreenSigned	bool	parameters.IsGreenSigned	no	Defaults to true.
CallbackUrl	Uri	n/a	no	Reserved; v1.1 hook.
CorrelationId	string	xPlaytest trace id	no	Echoed in status response.

✓ VERIFIED IN CODE

Verified: two required fields the source §5.1 omits — **PartnerId** & **PlaytestProductId**

Field	Source on xPlaytest	Req	Notes
PartnerId	PlaytestResponse.SellerId	yes	Offering owner forwarded to Partner Registry. Conflicts with §5.3 , which lists PartnerId as workflow-derived (<code>Id.Parse("PLAYTEST")</code>). Code has a <code>TODO : SellerId vs PublisherId unconfirmed (PlaytestIngestionJobParameters.cs:29-30; PlaytestIngestionPayloadBuilder.cs:108-110)</code> .
PlaytestProductId	parameters.PlaytestGeneratedXProductBigId	yes	Receiver builds offering id as <code>xpt{PlaytestProductId}</code> ; stable across republishes (<code>PlaytestIngestionJobParameters.cs:32-38</code>).

Receiver-side `JobParameters` also lists exactly these fields and validates them (`services.contentingestion PlaytestTitleIngestion.cs`). **Fix:** add both rows to §5.1, reconcile PartnerId with §5.3.

5.2 StoreAsset (xPlaytest fills)

This is the existing xCloud contract; xPlaytest must produce values that pass `StoreAsset.Validate()`.

Field	Source	Notes
StoreEntry.ProductId	PlaytestGeneratedXProductBigId	No Big Cat lookup from this id.
StoreEntry.Flags	const GAME	
StoreEntry.Name	PlaytestResponse.Name	
StoreEntry.Description	PlaytestResponse.Description	
StoreEntry.PublisherId	parameters.PublisherId	
StoreEntry.IsPrivateAudience	true	All playtests private.
StoreEntry.ParentProductIds	[PlaytestResponse.PartnerCenterProductId]	Traces back to source.
Markets	PlaytestConstants.DefaultMarkets	See verification below.
Platform	"WINDOWS.DESKTOP"	See platform mismatch below.
ContentId	PlaytestLifecycleState.PlaytestContentId	Must equal SourceId derived in the workflow.
XboxTitleId	caller-supplied (XOrc/parent product)	Validator throws if null/0 for games.
PackageFamilyName	PlaytestLifecycleState.PackageFamilyName	Required for PC.
AumId	"{PackageFamilyName}!App" (v1)	Required for PC games; real ApplicationId in v1.1.

⚠ REVIEW · DRIFT

Platform value mismatch (likely bug) — see §0

xbet sends `Platform = "WINDOWS.DESKTOP"` (`PlaytestConstants.cs:12; PlaytestIngestionPayloadBuilder.cs:94`), but the receiver throws `Unsupported platform` unless it is `ServerPlatform.PC` ("PC") or `Xbox` (`services.contentingestion StoreAsset.cs:35-46`). Map the platform to "PC" before sending, or widen the receiver.

⚠ REVIEW · DRIFT

Markets drift — not single-market US

The source §5.2 says `Markets = [Id.Parse("US")]`, but the builder sends `PlaytestConstants.DefaultMarkets` — ~87 markets (`PlaytestConstants.cs:61-72; PlaytestIngestionPayloadBuilder.cs:93`). Update the spec, or narrow the constant if single-market was intended for the playtest path.

✓ VERIFIED IN CODE

Verified receiver `StoreAsset.Validate()` rules

StoreEntry non-null + valid; Markets ≥ 1; ContentId non-empty; Platform non-empty and == PC or Xbox; if PC then PackageFamilyName required; if Flags == GAME then XboxTitleId required & nonzero, and PC also requires

AumID (`services.contentingestion ContentIngestion.Contracts.External/Workflow/StoreAsset.cs:24-61`). The xbet builder mirrors these offline (`PlaytestIngestionPayloadBuilder.cs:128-198`).

5.3 Derived inside PlaytestTitleIngestionWorkflow (xPlaytest does NOT send)

Field	Computation
PartnerId	<code>Id.Parse("PLAYTEST")</code> (<i>stale — superseded; PartnerId is now sent, see §5.1</i>)
OfferingId	<code>xpt-{ToShortId(PlaytestId)}</code> / receiver: <code>xpt{PlaytestProductId}</code>
TitleId	<code><alphanumOnly(Name)><PlaytestGeneratedXProductBigId></code>
SourceId	<code>StoreAsset.ContentId</code>
Audiences	<code>AllowedDnaGroups.Select(g => new XusAudience(g, AllowedSandboxId))</code>
AssetId	Generated by <code>AssetIngestionWorkflow.CreateAssetAsync</code>
Effective flight id for SUCU	Lex-smallest GUID from <code>AllowedDnaGroups</code> (§3.3)

✓ VERIFIED IN CODE

Verified: receiver consumes these as expected

`AllowedDnaGroups + AllowedSandboxId` become `new XusAudience(dnaGroup, AllowedSandboxId)`;
`ContentId` is the child asset source id; `XboxTitleId` flows into the Partner Registry `PlaytestRequest`
(`PlaytestTitleIngestionWorkflow.cs:81-90, 320-329`).

6. REST API Contract

6.1 POST /playtest/ingestion

Triggers asynchronous playtest ingestion; returns immediately with a jobId.

- **Auth:** S2S bearer, cross-tenant. Caller SPN in SAGE's AuthorizedClientAppls.
- **Request body:** PlaytestIngestionJobParameters (§5.1).
- **Response:** 202 Accepted with { jobId, statusUrl } (a Location header points to the status route).
- **Error codes:** 400 invalid StoreAsset; 401/403 auth; 409 duplicate in flight (rejoin polling); 5xx retryable.

6.2 GET /playtest/ingestion/{jobId}

Polls workflow state with the §3.10 backoff.

- **Auth:** Same S2S bearer.
- **Response 200:** { jobId, status, terminalState?, errors[], updatedAt, correlationId? }; status ∈ { Running, Streamable, Failed, InstallNotFound }.
- **Response 404:** Unknown jobId.

✓ VERIFIED IN CODE

Verified status shape differs from the spec's named states

The controller returns `OperationStatus` (defined in a referenced common package, not this repo) for both POST and GET (`WorkflowsControllerV3.cs:48-59`). Job success is `OfferingId != null` (`PlaytestTitleIngestion.cs JobState.IsSuccess()`); the "install not found" path is a fallback stage `NotifyInstallNotFoundAsync` (`PlaytestTitleIngestionWorkflow.cs:269-271`); the legacy status enum is { `NotStarted`, `Running`, `Succeeded`, `Failed`, `Conflict` } (`Legacy/JobStatus.cs:8-20`). The spec's {`Running`, `Streamable`, `Failed`, `InstallNotFound`} vocabulary should be reconciled with `OperationStatus`.

6.2.1 DELETE /playtest/ingestion/by-playtest/{playtestId}

Removes a published playtest's xCloud offering.

- **Behavior:** Sets `offering.PlayerAuthorizationOptions.AllowedDnaGroups = []`.
- **Response 200:** { playtestId, offeringId, tombstoneTime }; idempotent. **404:** never ingested.

⚠ REVIEW - DRIFT

DELETE / by-playtest route is doc-only — not implemented (see §0)

This route is in SAGE's OpenAPI but absent from SAGE route config and from the receiver `WorkflowsControllerV3` (`appsettings.xcloud.json:456-487`; `WorkflowsControllerV3.cs:48-59`). PT6 teardown depends on it. Either implement it (XC1/XC3) or remove it from the published contract until then.

7. Known Blockers

Blocker	Problem	Solution
Cross-tenant S2S	Playtest Service in MSFTGreen; SAGE disallows MSFTGreen.	Short term: PR #15715445 (Green→Corp). Long term: PME migration.
Partner Registry PR approvals	Each write = an ADO PR; SFI forbids self-approval.	Combine offering write + title attach into one PR via <code>BulkEditAsync</code> ; manual approval out-of-band.
XboxTitleId source	Builder hardcodes empty; validator rejects null/0 for games.	New resolver from XProduct <code>alternateIds[XboxTitleId]</code> / XOrc.
AumID for PC	Validator rejects null AumID on PC games.	v1 constant default; v1.1 real ApplicationId.

 REVIEW · STATUS

XboxTitleId blocker tracked but unresolved in the repo

The 'new resolver' does not yet exist: the builder still requires a caller-supplied `xboxTitleId` (`PlaytestIngestionPayloadBuilder.cs:50-58`) and `Blockers/XboxLiveIDs` is empty. Add owner + target milestone.

8. Resolved Decisions

⚠ REVIEW · MISSING

Section 8 is in the TOC but has no body in the source spec

The earlier reviewed draft had a complete 'Resolved Decisions' section dropped from Draft v2. **Recovered & corroborated by code:**

- **Cloud does not read XProduct directly.** xPlaytest assembles the full StoreAsset in the payload; the receiver consumes it through a playtest-specific path (no BigCat lookup) — matches the builder + receiver validate-then-ingest design.
- **Offerings are created only on publish**, from within XPackageWorkflow after PublishPlaytest hands off — not on every package upload.
- **All Playtests use the RETAIL sandbox** (default constant) (`PlaytestIngestionPayloadBuilder.cs:39, 114`).
- **Offering identity key is the stable product id** (`xpt{PlaytestProductId}`), not the per-snapshot `PublishedPlaytestId` — corroborated by the receiver contract and the Partner Registry commit 'Centralize playtest offering id... on PlaytestRequest'.

9. Open Questions & Recommendations

⚠ REVIEW · MISSING

Section 9 is in the TOC and cross-referenced (§9 Q3, §9 Q6) but has no body

Reconstructed list (renumber when finalized):

- **Q1 Terminal-state delivery.** Poll for v1 (§3.10); CallbackUrl in v1.1.
- **Q2 Canonical offering key.** Use the stable product id (now in §8 / §5.1).
- **Q3 End-date default & cap** (ref by §5.1, §12.1). 90-day pre-fill; no hard cap; soft warning at 180d.
- **Q4 PartnerId value.** SellerId vs PublisherId vs "PLAYTEST"; shipped code uses SellerId with a `TODO` — needs Partner Registry confirmation.
- **Q5 PC install-pool params** (§3.6). For v1 the receiver skips the PC install poll; confirm the real PC readiness check (`TODO` in code).
- **Q6 Seller audience-groups API** (§12.6). Adopt `GetGroupsBySellerAsync` for a DNA-group picker + clustering.
- **Q7 Platform contract** (new). Resolve the `WINDOWS.DESKTOP` vs `PC` mismatch (§0).

10. Success Criteria (P0)

⚠ REVIEW · MISSING

Section 10 has a heading but no body in the source spec

Recovered from the reviewed draft and the project brief's P0 list:

- Publishing with Cloud Streaming enabled **auto-triggers xCloud ingestion** (on publish only).
- A **private offering is created within seconds**, gated by `AllowedDnaGroups`, visible in DevApi Reader.
- The offering contains **exactly one title** bound to the Playtest product identity.
- **Authorized testers can launch and stream** via the shareable URL.
- **Audience / lifecycle changes propagate** (audience, expiry, republish, delete).
- **Unauthorized users learn nothing** — blocked cleanly, no metadata leak (Bayside).
- **A new build is ingested and the offering re-pointed** within minutes of upload.
- Offering creation can be **limited to an allow-list of sellers/titles** for internal testing.

11. Milestones & Foundational Demo

1. **Foundational Work ($\leq 6/7$)** — XC4 (AllowedDnaGroups on OfferingV2), XC5 (GsToken check), manual operator script, GMS propagation visible; acceptance: in-group tester streams, removed tester denied.
2. **Payload construction ($\leq 6/14$)** — PT2 + PT3: from in-memory state the builder produces a valid StoreAsset (incl. resolved XboxTitleId) + offering config; acceptance: validator-passing payload offline.
3. **SAGE connectivity ($\leq 6/30$)** — XC1 + XC2 + XC3 + XC6: a manual curl against SAGE produces a streamable offering. Midpoint Connect demo.
4. **Ingestion workflow integration ($\leq 7/14$)** — PT1 + PT4 + PT5: publishing a streaming playtest builds the payload, calls SAGE, returns a launch URL end-to-end.
5. **Lifecycle ($\leq 7/21$)** — PT6: audience / expiry / republish / delete reflect on the xCloud side.
6. **Soak + handoff ($\leq 7/28$)** — OpenAPI / partner docs; token-rotation + PR-approval runbook.
7. **Presentation week.**

✓ VERIFIED IN CODE

Verified milestone progress against shipped PRs / branches

Milestone 1 largely shipped: PR 15739964 (DNA groups on Offering edit), PR 15732852 (AllowedDnaGroups on PlayerAuthorizationOptions , XC4), PR 15738761 (login enforcement, XC5), PR 15773366 (new Playtest endpoint) ([PRProgress/*.md](#); [partnerregistry git log](#)). **Milestone 2 shipped:** the offline builder + contracts (PT2/PT3). **Milestone 3 partial:** SAGE routes (XC1) + receiver workflow (XC3) exist; XC6 PC-fork is a TODO in code. **Milestone 4–5 open:** PT4/PT5/PT6 + the DELETE route. Add a Status column to track this.

12. Next Steps

(Present in the body but missing from the TOC — see the TOC callout above.)

12.1 UI / Frontend (Bayside / Edgewater / Partner Center)

- Wire the launch URL into the Bayside offering-access path (no separate sign-in friction).
- Have GSSV return the playtest offerings a signed-in user is entitled to (“playtests available to you”).
- Add Partner Center affordances: streaming checkbox (XboxLive titles only), forced end-date picker (90d, \$9 Q3), DNA-group picker (\$12.6).

12.2 Authorization & PR-approval automation

- Automated pre-approval via parent product ID, removing the human round-trip.

12.3 Title eligibility: allow non-Xbox-Live titles

- Unblock not-yet-published titles (\$3.0 / \$3.12).

12.4 Cross-tenant identity & PME migration

Design Green → PME cross-tenant auth once xCloud finishes migrating (likely a Corp/PME proxy, mirroring the Greenbelt pattern).

12.5 Completion delivery — callbacks over polling

Switch to a CallbackUrl-driven signal once GSS ships cross-tenant messaging, eliminating poll load.

12.6 Catalog & clustering

- Bundle a seller’s playtests so a tester can “log into all of this seller’s private playtests.”
- Use Brian’s `GetGroupsBySellerAsync` (\$9 Q6).

12.7 Platform expansion

Xbox console support (Garrison / Bastion): Xbox-specific Title ID + package handling, console pools/ServerType fork, launch arguments, region / touch-control config, in-stream feedback. Real AumID plumbing in v1.1.

Appendix A: Glossary

- **DNA group:** GMS audience grouping resolved from Partner Center customer groups.
- **GMS:** Group Management Service. Resolves audience → FlightIds (DNA group GUIDs).
- **GsToken:** Game Streaming token; carries DnaGroups claims.
- **SAGE:** Service API Gateway; xCloud's cross-tenant ingress proxy and the single allow-listed entry point.
- **Ingestion:** xCloud's process of making a published build streamable (validate payload, register asset + package versions in SUCU, create/attach offering, drive first install).
- **SUCU:** Streaming Update Catalog; package-version resolution layer.
- **SFI:** Secure Future Initiative; forbids PR self-approval.
- **xpt-{id}:** Offering id prefix for playtest-derived offerings.

⚠ REVIEW - MISSING

Glossary is missing several terms the spec uses in italics / as jargon

Suggested additions: **Garrison** (current xPlaytest install/test endpoint), **Bastion** (console testing endpoint), **XProduct** (catalog product-document store), **XOrc** (sources title id / pfn / app id from the parent product), **Big Cat** (catalog pipeline that mints XboxTitleId for shipped titles), **Bayside** (Xbox web cloud-streaming endpoint), **XusAudience** ((dnaGroupGuid, sandboxId) wrapper), **OfferingV2** (Partner Registry offering contract), **TestX** (offering-level DNA-group gating pattern), **PME / Greenbelt** (Corp tenant + Green→PME proxy pattern), **AssetIngestionWorkflow** (existing xCloud asset-registration workflow), **ServerPlatform** (xCloud platform id; PC / Xbox).